Rajasthan Technical University, Kota (Raj.)

Project Report

Project Name-
CampusCare

Submitted By:

1. Roshan Jat	24EARAD143
2. Sufiyan Khan	24EARAD156
3. Sufiyan Nagori	24EARAD157
4. Sumit Kumar	24EARAD158
5. Divyanshu Geel	25EARAD702

Under the supervision of

Er. Ram Babu Buri

Assistant Professor

Session: 2025-26

Department of Artificial Intelligence & Data Science

ARYA COLLEGE OF ENGINEERING & I.T.

SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

CERTIFICATE OF APPROVAL

Department of Artificial Intelligence & Data Science

ARYA COLLEGE OF ENGINEERING & I.T., JAIPUR

(Affiliated to Rajasthan Technical University, Kota)

The project report entitled "CampusCare: Smart Campus Issue Management and Resolution System", submitted by Sufiyan Khan, Sufiyan Nagori, Roshan Jat, Sumit Yadav and Divyanshu Geel, is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Data Science for the academic session 2026–2027.

Project Guide / Evaluator	Head of the Department
_____ Er. Ram Babu Buri Assistant Professor	_____ Dr. Ashok Kumar Kajla Head of the Department AI & DS
Dept. of AI & DS, ACEIT	ACEIT, Jaipur

CERTIFICATE

Department of Artificial Intelligence & Data Science

ARYA COLLEGE OF ENGINEERING & I.T., JAIPUR

This is to certify that the project work embodied in this report entitled "CampusCare: Smart Campus Issue Management and Resolution System" has been carried out by Sufiyan Khan(24EARAD156), Sufiyan Nagori(24EARAD157), Roshan Jat(24EARAD143), Sumit Yadav(24EARAD158) and Divyanshu Geel(24EARAD702), students of Bachelor of Technology in Artificial Intelligence and Data Science, under the guidance of Er. Ram Babu Buri, Assistant Professor, Department of Artificial Intelligence & Data Science, Arya College of Engineering & I.T., Jaipur, during the academic session 2026–2027.

To the best of our knowledge, this report records the project work undertaken by the team. Final approval remains subject to review by the project guide and institute.

Date: _____
Place: Jaipur

_____
Er. Ram Babu Buri
Assistant Professor & Project Guide
Department of Artificial Intelligence & Data Science

DECLARATION

We, Sufiyan Khan, Sufiyan Nagori, Roshan Jat, Sumit Yadav and Divyanshu Geel, students of Bachelor of Technology in Artificial Intelligence and Data Science at Arya College of Engineering & I.T., Jaipur, declare that this report entitled "CampusCare: Smart Campus Issue Management and Resolution System" has been prepared as a record of our project work under the guidance of Er. Ram Babu Buri.

We further declare that implementation details, testing outcomes and results must correspond to the actual project. Any pending test observations, screenshots or institutional particulars have been marked for completion and verification before final submission.

Student Name	Responsibility	Signature
Sufiyan Khan	Team Lead; Authentication & Security; Core Infrastructure	_____
Sufiyan Nagori	Complaint Reporting; Tracking & History	_____
Roshan Jat	Staff Resolution; Notifications	_____
Sumit Yadav	Analytics, Feedback & Reports	_____
Divyanshu Geel	Admin Management	_____

Date: _____ Place: Jaipur

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Er. Ram Babu Buri, Assistant Professor, Department of Artificial Intelligence & Data Science, Arya College of Engineering & I.T., Jaipur, for his guidance and support during the planning and development of CampusCare.

We are thankful to the Head of the Department and the faculty members of the Department of Artificial Intelligence & Data Science for their academic guidance and for providing an environment that supported this project.

We acknowledge the efforts of every team member in designing, implementing, integrating and documenting the project modules. The use of Git and GitHub supported collaborative development and version control.

Finally, we thank our institute for the opportunity to undertake this project as part of our B.Tech academic work.

Sufiyan Khan | Sufiyan Nagori | Roshan Jat | Sumit Yadav | Divyanshu Geel

5th Semester, B.Tech in Artificial Intelligence & Data Science

Arya College of Engineering & I.T.,Jaipur

ABSTRACT

Educational campuses handle a large number of day-to-day issues related to Wi-Fi, electricity, laboratories, library, hostels, transport, security and furniture. These issues are usually reported through phone calls, paper applications or word of mouth, which leaves no record, no accountability and no way for the complainant to know the status of a request. As a result, complaints are delayed, duplicated or lost, and the administration has no data with which to improve its services.

This project presents CampusCare, a Smart Campus Issue Management and Resolution System developed as a Windows desktop application. The system provides a single digital channel through which students submit complaints with a category, location, priority, description and supporting attachments. Each complaint is automatically routed to the responsible department, assigned to a staff member by the administrator, and moved through a seven-stage lifecycle: submitted, assigned, accepted, in progress, resolved, closed and reopened. Every status change is stored with remarks, the person responsible and a timestamp, giving a complete audit history. Users receive in-app notifications on each change, students can rate the resolution, and administrators can view analytics by status, priority, department and category, and export reports as CSV files.

The application follows a layered architecture of controller, service, data-access and model layers. The user interface is built with Java 21, JavaFX and FXML, and the system uses PostgreSQL hosted on the cloud, accessed through JDBC, so that students, staff and administrators working on different computers see the same data. Role-based access control separates the student, staff and administrator views, passwords are protected with BCrypt hashing, and input validation is applied on all forms. The project is built with Maven and packaged as a standalone Windows executable with jpackage. The team followed a Git-based workflow with separate feature branches for each module.

CampusCare replaces informal complaint handling with a transparent, traceable and measurable process, and its analytics help campus authorities identify recurring problems and improve response quality.

Keywords: campus issue management, complaint tracking, JavaFX, PostgreSQL, role-based access, desktop application, workflow automation

TABLE OF CONTENTS

Section	Page
Front Matter	i–vii
Chapter 1: Introduction	
Chapter 2: Literature Survey	
Chapter 3: System Analysis	
Chapter 4: System Design	
Chapter 5: Implementation	
Chapter 6: Testing	
Chapter 7: Results and Discussion	
Chapter 8: Conclusion and Future Scope	
References	
Appendix A: SRS Reference	
Appendix B: User Manual	
Appendix C: Project Structure	

Note: Update page numbers in this table after final formatting and insertion of screenshots/diagrams in Microsoft Word.

LIST OF FIGURES

Figure 1.1: Overview of the problem and proposed solution

Figure 4.1: Layered architecture of CampusCare [diagram placeholder]

Figure 4.2: Use-case diagram [diagram placeholder]

Figure 4.3: Class diagram [diagram placeholder]

Figure 4.4: Entity-relationship diagram [diagram placeholder]

Figure 4.5: Complaint lifecycle state diagram [diagram placeholder]

Figure 4.6: Level 0 data-flow diagram [diagram placeholder]

Figure 4.7: Level 1 data-flow diagram [diagram placeholder]

Figure 7.1–7.12: Application screen placeholders

LIST OF TABLES

Table 1.1: Project objectives

Table 3.1: Functional requirements

Table 3.2: Non-functional requirements

Table 3.3: Software requirements

Table 3.4: Hardware requirements

Table 4.1: Module ownership

Table 4.2: Database tables and purpose

Table 4.3: Complaint status transitions

Table 5.1: Team contributions

Table 6.1: Test cases

CHAPTER 1

INTRODUCTION

1.1 Background

Educational institutions manage operational issues such as Wi-Fi connectivity, electrical faults, laboratory equipment, library facilities, hostel services, transport, security and furniture. These issues affect students and staff and require timely reporting, assignment and resolution. When reports are made through phone calls, paper applications or informal conversations, it can be difficult to maintain a central record, confirm ownership or track progress.

CampusCare was designed as a Windows desktop application that provides a common digital channel for campus issue management. It supports Student, Staff and Admin roles and maintains complaint information, assignments, status history, notifications and feedback. The application uses Java 21, JavaFX and FXML for its interface, JDBC for database access, and a shared PostgreSQL database hosted on Neon so authorised users on different computers can access common records.

1.2 Problem Statement

The main problem addressed by CampusCare is the absence of a unified and traceable process for reporting and resolving campus issues. Informal methods may provide limited visibility into whether a complaint has been received, which department is responsible, who is working on it, what actions have been taken and whether the student considers it resolved. The absence of consolidated records also limits the administration's ability to review recurring issues and service quality.

1.3 Objectives

1. Provide one digital channel for campus complaints.
2. Route complaints to the responsible department and support workload-aware staff assignment.
3. Provide status tracking and a complete complaint audit history.
4. Notify relevant users when complaint status changes.
5. Allow students to provide feedback and reopen resolved complaints.
6. Provide administrative analytics and CSV report export.
7. Enforce role-based access for students, staff and administrators.
8. Package the application for Windows with a bundled Java runtime.

1.4 Scope

The project covers one campus, three roles and seven departments. It targets Windows 10/11 and requires an internet connection to access the shared cloud database. Seeded master data consists of seven departments—IT, Electrical, Maintenance, Library, Hostel, Transport and Security—14 categories and 13 locations.

The Student module includes registration, login, complaint submission, tracking, history, notifications, feedback, reopening and profile functions. Staff can view assigned complaints, accept them, start work and resolve them with remarks. Admin users can assign and reassign complaints, manage users and master data, view analytics and export reports.

The current scope excludes a mobile application, email/SMS notifications and AI-based prioritisation. Attachments are limited to PNG, JPG, JPEG and PDF files up to 5 MB.

1.5 Proposed System Overview

A student submits a complaint with a title, description, category, location, priority and optional attachment. The complaint is recorded as SUBMITTED, a history entry is created, and notifications are generated for the student and active administrators. The system attempts to assign it to an active staff member in the relevant department who has the lowest workload. If no active staff member is available, the complaint remains SUBMITTED until an administrator assigns it.

The active assignee can accept the complaint, start work and resolve it with remarks. The student may submit a rating and optional comment after resolution; feedback automatically closes the complaint. A student may instead reopen a resolved complaint with a reason. All status changes pass through a workflow routine that updates the complaint timestamp and stores an audit-history record.

1.6 Organisation of the Report

Chapter 1 introduces the project. Chapter 2 presents the literature survey and identifies the required research gap, subject to insertion of the approved review material. Chapter 3 analyses the existing process, feasibility and requirements. Chapter 4 describes architecture, modules, data design and workflow. Chapter 5 explains implementation, security, database migration, packaging and collaboration. Chapter 6 defines the testing strategy and proposed test cases. Chapter 7 provides a screen walkthrough with screenshot placeholders. Chapter 8 presents the conclusion, limitations and future scope. References and appendices follow the chapters.

1.7 Project Identity

Item	Details
Project title	CampusCare – Smart Campus Issue Management and Resolution System
Application	Windows desktop client with shared cloud PostgreSQL
Programme	B.Tech Artificial Intelligence and Data Science
Session / Portal ID	2026–2027 / PBL2627-AI&DS-C-021
Guide	Er. Ram Babu Buri
Team	Sufiyan Khan (Team Lead), Sufiyan Nagori, Roshan Jat, Sumit Yadav, Divyanshu Geel
Institute	Arya College of Engineering & I.T., Jaipur
Repository	https://github.com/sufiyankhan73397-sudo/CampusCare

1.8 Chapter Summary

CampusCare was introduced as a structured desktop-based approach to campus complaint reporting, assignment and resolution. The chapter described its problem, objectives, scope, workflow and report organisation.

CHAPTER 2

LITERATURE SURVEY

2.1 Purpose of the Survey

The literature survey should establish the academic context for campus issue reporting, complaint management, workflow automation, status tracking, notifications, feedback and administrative analytics. A structured review normally compares the problem addressed by each source, its approach, reported findings and limitations.

2.2 Literature Review

[Literature review to be inserted]

The project brief supplied for this report contains an empty literature-review placeholder rather than the approved papers, comparison table or reference list. To comply with the instruction not to invent papers or citations, the paper-level review has not been fabricated. Insert the approved literature review and reference list provided by the team before final submission.

2.3 Comparative Analysis

The following comparison template is provided for the supplied research papers. All rows must be completed using source-supported information.

No.	Paper / Author / Year	Problem	Approach / Technology	Findings	Limitations / Relevance
1	[approved source]	[to be filled]	[to be filled]	[to be filled]	[to be filled]
2	[approved source]	[to be filled]	[to be filled]	[to be filled]	[to be filled]
3	[approved source]	[to be filled]	[to be filled]	[to be filled]	[to be filled]
4	[approved source]	[to be filled]	[to be filled]	[to be filled]	[to be filled]
5	[approved source]	[to be filled]	[to be filled]	[to be filled]	[to be filled]

2.4 Research Gap

[Research gap to be finalised after the approved literature review is inserted and compared.]

The project requirements identify practical issues that CampusCare aims to address: fragmented reporting, limited status visibility, unclear assignment ownership, incomplete history and limited consolidated analytics. These are project motivations, not a substitute for a literature-derived research gap. The final gap must be supported by the approved papers.

2.5 Relevance to CampusCare

The literature review should be used to justify relevant design choices, including role-based access, department-based routing, workload-aware assignment, a controlled lifecycle, audit history, notifications, feedback, reopening and administrative reports. A link between a research source and a design decision should be made only when the source directly supports it.

2.6 Summary

The structure for the literature survey is included, but the approved literature review and its reference list remain necessary to complete this chapter accurately.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Analysis of the Existing System

In the existing process described for this project, campus issues may be reported through phone calls, paper applications or word of mouth. These channels do not provide a single complaint register that students can use to track progress. Records may be lost, duplicated or delayed, and administrators may lack consolidated information for reviewing recurring problems.

3.2 Limitations of the Existing Process

- No single digital channel for all campus issues.
- Limited tracking of complaint status and history.
- Unclear accountability for assignment and resolution.
- Limited notification to students about progress.
- No centralised feedback and department-level analytics in the described process.
- Manual consolidation of complaint information for administrative review.

3.3 Proposed System

CampusCare provides a desktop interface for students, staff and administrators. It records complaint details, routes complaints by category to the relevant department, assigns work, manages permitted status changes and preserves history. In-app notifications communicate relevant events. Feedback, reopening, analytics and CSV exports support follow-up and administrative review.

3.4 Feasibility Study

3.4.1 Technical Feasibility

The selected stack consists of Java 21, JavaFX, FXML, JavaFX CSS, Maven, JDBC, jBCrypt and PostgreSQL hosted on Neon. The layered design separates the interface, business rules and database access. The Windows package is produced using jpackage with a bundled Java runtime. Deployment requires a working internet connection to reach the cloud database.

3.4.2 Economic Feasibility

The application is built using Java and open-source development tools. No verified project cost or hosting expenditure was supplied; therefore, a numerical cost-benefit claim is not made. Any cloud-plan costs, maintenance costs or institutional infrastructure costs should be added by the team if required.

3.4.3 Operational Feasibility

The three roles reflect the main participants in complaint handling: students report and track issues, staff resolve assigned issues, and administrators manage assignments and master data. The interface is designed for Windows desktop use. Because the database is cloud-hosted, network connectivity is required.

3.5 Software Requirements

Component	Requirement

Operating system	Windows 10 or Windows 11
Language / runtime	Java 21; runtime bundled in Windows package
User interface	JavaFX, FXML, JavaFX CSS
Build system	Maven
Database	PostgreSQL hosted on Neon
Connectivity	JDBC and internet connection
Password hashing	BCrypt using jBCrypt
Development environment	VS Code on Windows 11
Version control	Git and GitHub
Packaging	jpackage Windows application image

3.6 Hardware Requirements

A formal minimum hardware specification was not supplied in the project brief. The following should be completed after checking the actual development and target computers.

Hardware item	Specification
Processor	[minimum supported processor to be filled]
RAM	[minimum RAM to be filled]
Storage	[application and data storage requirement to be filled]
Display	[minimum display resolution to be filled]
Network	Internet connection required for cloud database access

3.7 Functional Requirements

ID	Requirement
FR-01	Users shall authenticate and access functions permitted for their role.
FR-02	Students shall register with validated name, email, phone and password fields.
FR-03	Students shall submit complaints with title, description, category, location, priority and optional attachment.
FR-04	The system shall store new complaints as SUBMITTED and create history and notifications.
FR-05	The system shall route complaints to the department associated with the category.
FR-06	The system shall assign to the least-loaded active staff member in the department when one is available.
FR-07	Administrators shall assign or reassign eligible complaints to active staff in the department.
FR-08	Only the active assignee shall accept, start work or resolve a complaint.
FR-09	Resolution shall require remarks between 5 and 500 characters.
FR-10	Students shall view their complaints, details, history and notifications.
FR-11	The complaint owner may reopen a RESOLVED complaint with a 5–500 character reason.
FR-12	The complaint owner may submit one rating from 1 to 5 and an optional comment up to 500 characters after resolution.
FR-13	Feedback submission shall automatically change the complaint status to CLOSED.
FR-14	Administrators shall manage users and master data, view analytics and export CSV reports.

3.8 Non-Functional Requirements

ID	Requirement
NFR-01	Security: role-based authorisation and BCrypt password hashes.
NFR-02	Validation: enforce stated field lengths, valid unique email, valid phone and password complexity.
NFR-03	Data integrity: multi-step database operations shall commit on success and roll back on failure.
NFR-04	Auditability: every status change shall record old status, new status, actor, remarks and timestamp.
NFR-05	Portability: the Windows package shall include a Java runtime.
NFR-06	Shared access: authorised users on separate computers shall use the same cloud database.
NFR-07	Attachment control: allow PNG, JPG, JPEG and PDF files up to 5 MB.
NFR-08	Maintainability: separate controller, service, DAO, model, utility, configuration and database responsibilities.

3.9 SRS Reference

The detailed Software Requirements Specification should be maintained as a separate project artefact. Reference: [SRS document title, version and date to be filled by team]. This report summarises the requirements supplied in the project brief.

3.10 Summary

The analysis identifies the shortcomings of informal complaint handling and describes how CampusCare addresses them through centralised records, controlled workflows, notifications and reports. Hardware minimums and formal SRS metadata remain to be verified.

CHAPTER 4

SYSTEM DESIGN

4.1 Design Overview

CampusCare uses a layered architecture. JavaFX and FXML provide the presentation layer. Controllers handle user actions and coordinate with services. Services implement business rules, including complaint transitions and assignment. DAOs execute JDBC operations against PostgreSQL. Model classes represent application entities, while utility and configuration packages support validation, navigation, database settings and common operations.

[Figure 4.1: Layered architecture — JavaFX/FXML View → Controller → Service → DAO → JDBC → PostgreSQL. Supporting packages: Model, Util, Config and Database.]

4.2 Architectural Layers

Layer	Responsibility
View / UI	FXML layouts and JavaFX CSS for login, dashboards, forms and data views.
Controller	Handles UI events, collects input and invokes services.
Service	Implements validation, authorisation and complaint workflow rules.
DAO	Encapsulates SQL statements and maps database rows to models.
JDBC / Database	Provides transactional database access to PostgreSQL.
Model	Represents users, departments, categories, locations, complaints, assignments, history, notifications and feedback.
Util / Config	Shared helpers and configuration, including database connection settings.

4.3 Module Description and Ownership

Module	Functions	Owner
M1 Authentication & Security	Registration, login, password hashing, role checks and logout.	Sufiyan Khan
M2 Complaint Reporting	Complaint form, field validation, attachment checks and submission.	Sufiyan Nagori
M3 Complaint Tracking & History	Student complaint list, details, status and audit timeline.	Sufiyan Nagori
M4 Staff Resolution	Assigned queue, accept, start work and resolve with remarks.	Roshan Jat
M5 Notification	Create and display in-app notifications for workflow events.	Roshan Jat
M6 Analytics, Feedback & Reports	Feedback, analytics by status/priority/department/category and CSV export.	Sumit Yadav
M7 Admin Management	User and master-data management, complaint assignment and reassignment.	Divyanshu Geel
Core / Infrastructure	Database connection and initialisation, configuration, navigation and packaging.	Sufiyan Khan

4.4 Database Design

The application uses a shared PostgreSQL database hosted on Neon. The following ten tables form the supplied logical data model. Exact SQL types, constraints and indexes should be checked against the final migration scripts before submission.

Table	Purpose / Principal fields
departments	department_id, department_name, description, status — department master data.
users	user_id, name, email, password, phone, role, department_id, status, created_at — student, staff and admin accounts.
categories	category_id, category_name, department_id — complaint categories and department mapping.
locations	location_id, location_name, building, floor, description — campus location master data.
complaints	complaint_id, user_id, category_id, location_id, title, description, priority, status, created_at, updated_at, resolved_at — complaint record.
staff_assignments	assignment_id, complaint_id, staff_id, assigned_by, assigned_at, status — assignment history and active assignment state.
complaint_history	history_id, complaint_id, updated_by, old_status, new_status, remarks, updated_at — audit trail.
attachments	attachment_id, complaint_id, file_name, file_path, uploaded_by, uploaded_at — attachment metadata and local path.
notifications	notification_id, user_id, complaint_id, message, is_read, created_at — in-app notification records.
feedback	feedback_id, complaint_id, user_id, rating, comments, created_at — student rating and comment.

4.5 Data Dictionary Notes

The identifiers ending in "_id" act as entity identifiers. Relationship fields connect complaints to their owner, category and location; categories to departments; staff assignments to complaints and staff users; history and attachments to complaints; and notifications and feedback to the relevant users and complaints. The final schema should enforce appropriate primary keys, foreign keys, uniqueness and valid-value constraints.

4.6 Complaint Lifecycle

The seven defined statuses are SUBMITTED, ASSIGNED, ACCEPTED, IN_PROGRESS, RESOLVED, CLOSED and REOPENED. The priority values are NORMAL, MEDIUM, HIGH and CRITICAL. The status lifecycle is enforced through a common workflow routine that updates status and updated_at and records the transition in complaint_history.

From	To	Actor	Condition
(new)	SUBMITTED	Student	complaint submitted
SUBMITTED, ASSIGNED, ACCEPTED, REOPENED	ASSIGNED	Admin	active staff of the department
SUBMITTED	ASSIGNED	System (auto)	least-loaded active staff exists
ASSIGNED	ACCEPTED	Staff (assignee)	optional remarks
ASSIGNED, ACCEPTED	IN_PROGRESS	Staff (assignee)	optional remarks
IN_PROGRESS	RESOLVED	Staff (assignee)	remarks 5-500 characters
RESOLVED	REOPENED	Student (owner)	reason 5-500 characters

REOPENED	ASSIGNED	System (auto)	last staff, else least-loaded
RESOLVED	CLOSED	System (auto)	feedback submitted

4.7 Business Rules

- Complaint title must contain 5–100 characters; description must contain 10–1000 characters.
- Name must contain 3–60 characters; email must be valid and unique; phone must be valid.
- Password must be at least 8 characters and include one uppercase letter, one lowercase letter and one digit.
- Only the active assignee may act on an assigned complaint. Accept is allowed from ASSIGNED; starting work is allowed from ASSIGNED or ACCEPTED; resolving is allowed only from IN_PROGRESS.
- Feedback is allowed only for the complaint's own student, only in RESOLVED state, and once per complaint. Rating must be 1–5; comment is optional and limited to 500 characters.
- Reopening is allowed only to the complaint owner from RESOLVED and requires a 5–500 character reason.
- Reopening clears resolved_at and triggers reassignment to the last staff member who worked on it, or the least-loaded eligible staff member if needed.
- Attachments may be PNG, JPG, JPEG or PDF, up to 5 MB. Files are saved under random names in a local folder; only the path is stored in the database.
- First-run master data includes 7 departments, 14 categories and 13 locations. A default admin and one demo staff account per department are created; passwords are BCrypt hashes.

4.8 Use-Case Diagram

[Figure 4.2: Use-case diagram placeholder. Student use cases: register, login, submit complaint, view complaints, view history, receive notifications, give feedback, reopen complaint and logout. Staff use cases: login, view assigned complaints, accept, start work, resolve and logout. Admin use cases: login, assign/reassign complaints, manage users and master data, view analytics, export CSV reports and logout.]

4.9 Class Diagram

[Figure 4.3: Class diagram placeholder. Suggested classes correspond to the supplied entities and layers: User, Department, Category, Location, Complaint, StaffAssignment, ComplaintHistory, Attachment, Notification, Feedback; controllers; services; DAOs; and database/configuration utilities. Confirm class names against the source code.]

4.10 Entity-Relationship Diagram

[Figure 4.4: ER diagram placeholder. Show the relationships between departments and categories/users, users and complaints, complaints and locations/categories, complaints and assignments/history/attachments/notifications/feedback. Confirm cardinalities against the final schema.]

4.11 State Diagram

[Figure 4.5: State diagram placeholder. Represent the transitions in Table 4.3, including feedback-driven closure and reopening followed by reassignment.]

4.12 Data-Flow Diagrams

[Figure 4.6: Level 0 DFD placeholder. External entities Student, Staff and Admin exchange authentication, complaint, assignment, resolution, feedback and report data with CampusCare.]

[Figure 4.7: Level 1 DFD placeholder. Processes: authentication and access control; complaint reporting; assignment and resolution; tracking/history and notifications; feedback and closure/reopening; admin management and analytics. Data stores correspond to the ten database tables.]

4.13 Summary

This chapter documented the layered design, seven modules, ten-table logical data model, complaint workflow and diagram specifications. Diagram placeholders should be replaced with diagrams aligned with the actual implementation.

CHAPTER 5

IMPLEMENTATION

5.1 Technology Stack and Rationale

Technology	Role in CampusCare
Java 21	Primary programming language and runtime target.
JavaFX / FXML / JavaFX CSS	Desktop interface, declarative layouts and styling.
Maven	Dependency and build management.
JDBC	Relational database connectivity.
PostgreSQL on Neon	Shared cloud database for multiple computers.
jBCrypt / BCrypt	Password hashing.
Git / GitHub	Version control, branches and pull requests.
jpackage	Creates a Windows application image with a bundled Java runtime.
VS Code / Windows 11	Development environment and operating system.

5.2 Project Structure

The implementation follows a layered package organisation. The names below describe the intended responsibilities; replace or refine them to match the actual repository if the package layout differs.

```
CampusCare/
├── pom.xml
├── src/main/java/[base package]/
│   ├── controller/   – JavaFX event handlers
│   ├── service/      – business rules and workflow
│   ├── dao/          – JDBC queries and persistence
│   ├── model/        – entity/domain classes
│   ├── util/         – shared helpers and validation
│   ├── config/       – application and database configuration
│   └── database/     – connection and initialisation logic
├── src/main/resources/
│   ├── fxml/         – JavaFX layouts
│   ├── css/          – JavaFX stylesheets
│   └── [other resources]
├── [local attachment storage directory]
└── [jpackage output directory]
```

5.3 Authentication and Security

The authentication module validates user credentials and applies role-based access control. Passwords are stored as BCrypt hashes rather than plaintext. Registration validation enforces name length, email validity and

uniqueness, phone validity, and password requirements. Controllers should not be treated as the only security boundary: service-level checks must confirm the user's role and ownership before sensitive operations.

Database queries should use parameterised JDBC statements. Database credentials are kept in a configuration file that is not committed to GitHub. The final package must be reviewed to ensure that secrets are not unintentionally exposed in a distributable configuration. The supplied brief states that database settings are bundled inside the application; any credentials or secrets placed in a desktop package should be treated as potentially recoverable and protected accordingly.

5.4 Complaint Submission and Workflow

On submission, the application validates required fields and optional attachment type and size. It records the complaint as SUBMITTED, writes a history entry and creates notifications for the student and active administrators. Category-to-department mapping determines the responsible department. When active staff exist in that department, the system selects the staff member with the lowest workload; otherwise, manual administrator assignment is required.

The workflow routine centralises status changes and audit-history creation. Multi-step database operations run in a transaction, committing on success and rolling back on failure. This protects related changes—such as assignment, status update, history insertion and notification creation—from being partially applied when an operation fails.

5.5 Staff Resolution and Notifications

Only the active assignee can accept a complaint, start work or resolve it. Accept is permitted from ASSIGNED; work can begin from ASSIGNED or ACCEPTED; resolution is permitted only from IN_PROGRESS and requires remarks between 5 and 500 characters. Notifications are created for relevant users as the workflow changes. The precise wording and screen placement should be confirmed against the implemented UI.

5.6 Feedback, Reopening and Analytics

A complaint's own student may submit feedback once, only when the complaint is RESOLVED. The rating is from 1 to 5 and the optional comment is limited to 500 characters. Feedback automatically changes the status to CLOSED and notifies the student and assigned staff member. Alternatively, the student may reopen a resolved complaint with a 5–500 character reason. Reopening clears resolved_at, notifies the student and administrators, and triggers reassignment.

The admin analytics module provides views by status, priority, department and category, and allows CSV report export. Actual chart types, filters and exported fields should be described in the final report only after they are checked in the application.

5.7 SQLite-to-PostgreSQL Migration

The project initially used a local SQLite database. During development, the team identified that each computer maintained a separate local database; a complaint entered on one laptop was therefore not visible on another. To provide shared records across computers, the design was migrated to PostgreSQL hosted on Neon.

The migration required changing the JDBC driver and connection class, adapting table-creation SQL (including AUTOINCREMENT to SERIAL and date-default syntax), and moving connection credentials to a

configuration file excluded from GitHub. This was a design decision driven by the multi-computer use case, not merely a change in database preference. The application requires internet connectivity to access the cloud database.

5.8 Packaging as a Windows Application

The project uses jpackage to create a Windows application image with the Java runtime bundled. This is intended to let the evaluator run the application without installing a separate JDK or IDE. The final deliverable should include the generated application image or installer, any required local attachment directory, and instructions for configuring or validating database connectivity. Verify the exact output name and whether the final deliverable is an application image, installer or executable launcher.

5.9 Git and Team Workflow

The team used Git and GitHub for version control. The main branch represents the final stable version, develop is the integration branch, and feature branches include feature/student, feature/staff, feature/admin-dashboard and feature/admin-mgmt. Pull Requests are merged into develop. The supplied workflow also specifies one commit per member per day.

5.10 Team Contribution

Member	Assigned module / area	Work responsibility
Sufiyan Khan	M1 Authentication & Security; Core / Infrastructure	Authentication and security; database connection and initializer; configuration, navigation and packaging.
Sufiyan Nagori	M2 Complaint Reporting; M3 Tracking & History	Complaint submission, validation, complaint list, details and audit history.
Roshan Jat	M4 Staff Resolution; M5 Notification	Staff workflow actions and in-app notification functionality.
Sumit Yadav	M6 Analytics, Feedback & Reports	Feedback workflow, analytics and CSV reporting.
Divyanshu Geel	M7 Admin Management	Admin user management, master data and complaint assignment management.

5.11 Summary

The implementation chapter describes the technology stack, layered organisation, workflow, security controls, database migration, packaging and contribution allocation. Final package details and the actual repository structure should be verified before submission.

CHAPTER 6

TESTING AND QUALITY ASSURANCE

6.1 Testing Objectives

Testing is required to determine whether CampusCare follows the stated validation rules, role permissions, complaint transitions, assignment behaviour, audit-history requirements and database transaction boundaries. No executed test results were supplied with the project brief. Accordingly, the actual-result and status columns below are intentionally left for the team to complete after running the tests.

6.2 Test Strategy

- Unit-level checks: validate field rules, priority/status handling, assignment selection and service-level workflow conditions.
- Functional testing: verify registration, login, complaint submission, tracking, staff actions, feedback, reopening and administration.
- Role-based access testing: confirm that Student, Staff and Admin users can perform only permitted operations.
- Validation testing: check field lengths, malformed/duplicate email, password policy, invalid phone and attachment type/size.
- Database integration testing: verify persistence, history, notifications and transaction rollback for multi-step operations.
- Cross-computer integration: use two separate computers connected to the shared PostgreSQL database and confirm that a complaint created on one is visible to an authorised user on the other.
- Packaging testing: launch the generated Windows package on a target computer without a separately installed JDK and verify database connectivity.

6.3 Test Cases

ID	Module	Test description	Input / precondition	Expected result	Actual result	Status
TC-01	Authentication	Register with valid fields	Valid name, unique email, valid phone, compliant password	Account is created successfully	[to be filled]	[to be filled]
TC-02	Authentication	Reject short name	Name under 3 characters	Validation error; account not created	[to be filled]	[to be filled]
TC-03	Authentication	Reject long name	Name over 60 characters	Validation error; account not created	[to be filled]	[to be filled]
TC-04	Authentication	Reject duplicate email	Email already registered	Duplicate email is rejected	[to be filled]	[to be filled]
TC-05	Authentication	Reject weak password	Password lacks required character classes	Validation error; account not created	[to be filled]	[to be filled]
TC-06	Authentication	Login with valid credentials	Correct email and password	User enters role-appropriate view	[to be filled]	[to be filled]
TC-07	Authentication	Reject invalid credentials	Incorrect password	Login is denied	[to be filled]	[to be filled]
TC-08	Access Control	Student attempts admin action	Student session invokes admin function	Operation is denied	[to be filled]	[to be filled]
TC-09	Complaint Reporting	Submit valid complaint	Valid title, description,	Complaint stored as SUBMITTED	[to be filled]	[to be filled]

			category, location, priority			
TC-10	Complaint Reporting	Reject short title	Title under 5 characters	Validation error; not submitted	[to be filled]	[to be filled]
TC-11	Complaint Reporting	Reject long title	Title over 100 characters	Validation error; not submitted	[to be filled]	[to be filled]
TC-12	Complaint Reporting	Reject short description	Description under 10 characters	Validation error; not submitted	[to be filled]	[to be filled]
TC-13	Complaint Reporting	Reject oversized description	Description over 1000 characters	Validation error; not submitted	[to be filled]	[to be filled]
TC-14	Attachments	Upload permitted image	PNG/JPG/JPEG within 5 MB	Attachment accepted and path recorded	[to be filled]	[to be filled]
TC-15	Attachments	Reject unsupported file	Unsupported extension	Attachment rejected	[to be filled]	[to be filled]
TC-16	Attachments	Reject file over 5 MB	Valid extension but size over limit	Attachment rejected	[to be filled]	[to be filled]
TC-17	Assignment	Auto-assign with active staff	Complaint department has active staff	Least-loaded eligible staff is assigned	[to be filled]	[to be filled]
TC-18	Assignment	No active staff available	No active staff in complaint department	Complaint remains SUBMITTED for admin assignment	[to be filled]	[to be filled]
TC-19	Admin Management	Admin assigns complaint	Active staff belongs to correct department	Status becomes ASSIGNED; relevant users notified	[to be filled]	[to be filled]
TC-20	Staff Resolution	Accept assigned complaint	Active assignee; status ASSIGNED	Status becomes ACCEPTED	[to be filled]	[to be filled]
TC-21	Staff Resolution	Reject action by non-assignee	Different staff user attempts action	Operation is denied	[to be filled]	[to be filled]
TC-22	Staff Resolution	Start work from ASSIGNED	Active assignee	Status becomes IN_PROGRESS	[to be filled]	[to be filled]
TC-23	Staff Resolution	Resolve with valid remarks	IN_PROGRESS and 5–500 character remarks	Status becomes RESOLVED; history is recorded	[to be filled]	[to be filled]
TC-24	Staff Resolution	Resolve with short remarks	Remarks under 5 characters	Validation error; status unchanged	[to be filled]	[to be filled]
TC-25	Tracking & History	View complaint history	Complaint with multiple transitions	History shows actor, old/new status, remarks and time	[to be filled]	[to be filled]
TC-26	Feedback	Submit valid feedback	Owner, RESOLVED complaint, rating 1–5	Feedback stored; status becomes CLOSED	[to be filled]	[to be filled]
TC-27	Feedback	Submit duplicate feedback	Feedback already exists for complaint	Second submission is rejected	[to be filled]	[to be filled]
TC-28	Feedback	Non-owner submits feedback	Different student account	Operation is denied	[to be filled]	[to be filled]
TC-29	Reopening	Reopen resolved complaint	Owner and 5–500 character reason	Status becomes REOPENED; reassignment begins	[to be filled]	[to be filled]
TC-30	Reopening	Reopen with short reason	Reason under 5 characters	Validation error; status unchanged	[to be filled]	[to be filled]
TC-31	Notifications	Verify submission notifications	Student submits complaint	Student and active admins receive in-app notifications	[to be filled]	[to be filled]
TC-32	Analytics / Reports	Export CSV report	Admin requests report	CSV is generated with selected report data	[to be filled]	[to be filled]
TC-33	Transactions	Force multi-step operation failure	Simulated failure during transaction	Changes roll back; no partial workflow update	[to be filled]	[to be filled]
TC-34	Cloud Integration	Create complaint on computer A	Computer A connected to shared database	Record is stored in cloud database	[to be filled]	[to be filled]

TC-35	Cloud Integration	View complaint on computer B	Authorised user on computer B	Same record is visible from shared database	[to be filled]	[to be filled]
TC-36	Packaging	Launch packaged application	Windows target without separate JDK	Application launches with bundled runtime	[to be filled]	[to be filled]

6.4 Defect Recording

Any failure should be recorded with the test ID, environment, reproduction steps, observed behaviour, expected behaviour, severity, assigned owner and retest result. The status field should use the team's agreed terms, such as Pass, Fail, Blocked or Not Run. Do not mark tests as passed until they have actually been executed.

6.5 Summary

A test plan and 36 test cases are provided. The team must execute them on the final build and fill in actual results and statuses. No test outcomes are asserted in this report.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Overview

This chapter is intended to document the application screens and explain the workflow visible to each role. Actual screenshots were not supplied with the project materials, so the placeholders below must be replaced with genuine screenshots from the final application. The descriptions are based on the specified features and should be adjusted if the final UI differs.

7.2 Login Screen

[Screenshot 7.1: Login screen]

Shows the login interface through which an existing user enters credentials. The final screenshot should demonstrate the actual fields, validation messages and navigation.

7.3 Registration Screen

[Screenshot 7.2: Registration screen]

Shows student registration and the applicable name, email, phone and password validation.

7.4 Student Dashboard

[Screenshot 7.3: Student dashboard]

Shows the student's main workspace and available complaint-related navigation.

7.5 Submit Complaint

[Screenshot 7.4: Submit complaint form]

Shows title, description, category, location, priority and optional attachment fields.

7.6 My Complaints

[Screenshot 7.5: My complaints list]

Shows the complaints associated with the logged-in student and their current statuses.

7.7 Complaint Details and History

[Screenshot 7.6: Complaint details and history]

Shows complaint information and the recorded status timeline, including actor, remarks and timestamp where provided by the UI.

7.8 Staff Dashboard

[Screenshot 7.7: Staff dashboard]

Shows the staff member's assigned complaint queue and permitted actions.

7.9 Admin Dashboard

[Screenshot 7.8: Admin dashboard]

Shows administrative navigation and available management functions.

7.10 Analytics

[Screenshot 7.9: Analytics dashboard]

Shows analytics by status, priority, department and category. Insert the actual chart view.

7.11 Feedback

[Screenshot 7.10: Feedback screen]

Shows the rating and optional comment form available after a complaint is resolved.

7.12 User Management

[Screenshot 7.11: User management]

Shows the actual administrator interface for user management.

7.13 Notifications

[Screenshot 7.12: Notifications screen]

Shows in-app notification records and read/unread presentation if implemented.

7.14 Discussion

The specified workflow aims to improve traceability by linking complaint submission, assignment, resolution, history, notification and feedback. Shared PostgreSQL storage is intended to make the same complaint data available to authorised users on different computers. Whether these objectives were achieved in the final build should be confirmed through the tests in Chapter 6 and through genuine screenshots.

7.15 Evidence Required Before Submission

- Insert screenshots from the final packaged application, not mock-ups.
- Confirm that the displayed account roles and features match the running build.
- Include the test execution date and environment where required by the college.
- Fill in actual test outcomes and resolve any failed or blocked cases.
- Verify CSV exports and cross-computer data visibility before describing them as demonstrated results.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

CampusCare was designed as a Windows desktop application to provide a centralised and traceable process for campus issue reporting and resolution. The specified system brings student complaint submission, department routing, staff assignment, status tracking, audit history, in-app notifications, feedback, reopening and administrative analytics into one workflow.

The application design uses Java 21, JavaFX, FXML, JDBC and a shared PostgreSQL database hosted on Neon. The layered architecture separates UI controllers, business services, data-access objects and model classes. BCrypt password hashing, input validation, role-based access and database transactions are specified to support security and consistency. Migration from local SQLite to cloud PostgreSQL addresses the requirement for multiple computers to access shared complaint records.

The report defines the architecture, data model, workflow rules, implementation approach and test plan. The literature review, diagram images, application screenshots, hardware minimums and actual test results remain items for the team to verify or insert before final submission.

8.2 Limitations

- The desktop application requires an internet connection to access the shared cloud database.
- Email and SMS notifications are outside the current scope; notifications are in-app.
- AI-based prioritisation is not implemented in the described scope.
- The application is currently scoped to one campus and Windows 10/11 desktop use.
- Actual test outcomes and screenshot evidence must be added from the final build.
- Attachments are stored in a local folder while only their paths are stored in the database; deployment and backup procedures should account for the location of those files.

8.3 Future Scope

- Develop an Android or other mobile application for convenient complaint submission and tracking.
- Add email and SMS notifications for important workflow changes.
- Investigate AI-based complaint prioritisation after suitable requirements, data and evaluation criteria are established.
- Introduce SLA tracking to monitor target response and resolution times.
- Extend reporting and trend analysis based on validated operational data.
- Define a robust attachment backup and migration approach for multi-computer use.

8.4 Final Summary

CampusCare provides a defined foundation for digital campus issue management. Before formal submission, the team should complete the literature chapter using its approved references, replace diagram and screenshot placeholders, verify the packaged build, and record actual test outcomes.

REFERENCES

The project brief did not contain the approved literature review or research-paper reference list. No research papers have therefore been invented. The following official technology documentation references are provided for the technologies named in the project specification; verify the access date and exact version used before submission.

[1] Oracle, “Java Documentation.” [Online]. Available: https://docs.oracle.com/en/java/ . [Accessed: 13-Oct-2026].

[2] OpenJFX, “JavaFX Documentation.” [Online]. Available: https://openjfx.io/openjfx-docs/ . [Accessed: 13-Oct-2026].

[3] PostgreSQL Global Development Group, “PostgreSQL Documentation.” [Online]. Available: https://www.postgresql.org/docs/ . [Accessed: 13-Oct-2026].

[4] Apache Maven Project, “Maven Documentation.” [Online]. Available: https://maven.apache.org/guides/ . [Accessed: 13-Oct-2026].

[5] jBCrypt, “jBCrypt: BCrypt hashing for Java.” [Online]. Available: https://www.mindrot.org/projects/jBCrypt/ . [Accessed: 13-Oct-2026].

[6] Oracle, “The jpackage Command.” Java Platform, Standard Edition Tools Reference. [Online]. Available: https://docs.oracle.com/en/java/javase/21/docs/specs/man/jpackage.html . [Accessed: 13-Oct-2026].

[Add the approved literature-review references here in IEEE style. Do not add a source unless it was actually consulted and is included in the approved review or official technology documentation.]

APPENDIX A

SOFTWARE REQUIREMENTS SPECIFICATION (SRS) REFERENCE

The report summarises the supplied functional and non-functional requirements. Attach or cross-reference the final approved SRS here.

SRS field	Value
Document title	[to be filled]
Version	[to be filled]
Approval date	[to be filled]
Prepared by	CampusCare project team
Repository	https://github.com/sufiyankhan73397-sudo/CampusCare

APPENDIX B

USER MANUAL

B.1 Starting the Application

9. Run the packaged CampusCare application on a supported Windows 10/11 computer.
10. Ensure that the computer has internet access to reach the shared PostgreSQL database.
11. Sign in with the appropriate account. Default admin and demo staff accounts are created on first run; obtain the actual credentials from the authorised project configuration or administrator.
12. Do not publish or share default credentials. Change or replace demonstration credentials according to the project's security procedure.

B.2 Student Workflow

13. Register using a name, unique email, valid phone number and password that meets the stated complexity rules.
14. Log in with the registered account.
15. Open the complaint submission screen and enter a title, description, category, location and priority.
16. Optionally attach a PNG, JPG, JPEG or PDF file no larger than 5 MB.
17. Submit the complaint and note its status. The initial status is SUBMITTED.
18. Use My Complaints to open a complaint and review its details and history.
19. Review in-app notifications for relevant updates.
20. When the complaint is RESOLVED, submit a rating from 1 to 5 and an optional comment up to 500 characters to close it, or reopen it with a reason of 5–500 characters if needed.
21. Log out after completing the session.

B.3 Staff Workflow

22. Log in using the authorised staff account.
23. Review the assigned complaint queue.
24. Open a complaint for which the account is the active assignee.
25. Accept the complaint from ASSIGNED if appropriate; acceptance is optional before starting work.
26. Start work from ASSIGNED or ACCEPTED.
27. Resolve the complaint from IN_PROGRESS and provide remarks between 5 and 500 characters.
28. Review notifications and log out after the session.

B.4 Admin Workflow

29. Log in with an authorised admin account.
30. Review new complaints and assign or reassign complaints to active staff belonging to the responsible department.
31. Manage users and master data using the available admin screens.
32. Review analytics by status, priority, department and category.
33. Export the required report as a CSV file.
34. Review notifications and log out after the session.

B.5 Troubleshooting

Issue	Recommended check
Cannot log in	Check credentials, account status and the configured role.
Database connection error	Check internet connectivity and the approved database configuration; do not expose credentials in screenshots or public repositories.
Complaint not auto-assigned	Confirm that active staff exist in the complaint's department; otherwise request admin assignment.
Attachment rejected	Use PNG/JPG/JPEG/PDF and ensure the file is no larger than 5 MB.
Action is unavailable	Check the complaint status, active assignment and role/ownership conditions.

APPENDIX C

KEY CODE STRUCTURE AND REPOSITORY

The codebase is organised around controller, service, DAO, model, utility, configuration and database responsibilities. The actual class and package names should be confirmed against the checked-in repository before final submission.

```
src/main/java/[base package]/controller/
src/main/java/[base package]/service/
src/main/java/[base package]/dao/
src/main/java/[base package]/model/
src/main/java/[base package]/util/
src/main/java/[base package]/config/
src/main/java/[base package]/database/
src/main/resources/fxml/
src/main/resources/css/
pom.xml
Configuration file: [actual path; exclude secrets from public version control]
Packaged output: [actual jpackage output path]
```

C.1 Repository

GitHub repository: https://github.com/sufiyankhan73397-sudo/CampusCare

C.2 Final Submission Checklist

☐ Replace [college name] or any remaining placeholders if required by the department.

☐ Insert the approved literature review table, research gap and associated references.

☐ Replace all diagram placeholders with diagrams consistent with the implementation.

☐ Insert genuine screenshots from the final packaged application.

☐ Run the test cases and fill Actual Result and Status columns accurately.

☐ Verify the final database schema, package names, installer/image name and deployment configuration.

☐ Update the table of contents and page numbers in Word.